CS 498 — Cloud and Cluster Data Management

FoundationDB: A Distributed Unbundled Transactional Key Value Store

Primary Source:

- Zhou, J., Xu, M., Shraer, A., et al. (2021). "FoundationDB: A Distributed Unbundled Transactional Key Value Store." SIGMOD 2021. https://doi.org/10.1145/3448016.3457559

What You Will Learn Today

01

Motivation & Background

Why cloud systems need both scalability and strong consistency; the NoSQL-ACID gap.

02

FDB Architecture

Unbundled design: Control Plane, Transaction System, Log System, Log System, Storage System.

03

Transaction Management

End-to-end read/write processing, OCC + MVCC, conflict detection, logging.

04

Fault Tolerance & Recovery

Failure detection, recovery process, replication strategies.

05

Geo-Replication

Cross-region design, LogRouters, failover mechanisms.

06

Evaluation

Production measurements at Apple, scalability tests, reconfiguration time.

Motivation: The Scalability-Consistency Dilemma

The Scale Requirement

- Billions of concurrent users
- Petabytes to exabytes of data
- Millions of requests per second
- Geographic distribution across regions

The Consistency Gap

- NoSQL systems (Cassandra, DynamoDB, MongoDB) scaled by sacrificing:
  - Transactional semantics → eventual consistency
- Developers must reason about concurrent update interleaving
- Application-level conflict resolution required

"NoSQL systems retrofitted ACID transactions later (Spanner, DynamoDB Transactions), but FDB was designed with strict serializability from the start."

FoundationDB: Core Philosophy

> ***"Foundational set of building blocks required to build higher-level distributed systems"***

Ordered KV Store

Natively supports multi-key strictly serializable transactions across the entire key space.

Modular Design

No schema, no query language, no secondary indices — these are built as layers on top.

Layered Architecture

Higher-level systems (relational, graph, document) built as stateless layers on FDB.

Example Layers

- **Record Layer** — Relational database capabilities (used by CloudKit at Apple)
- **JanusGraph** — Graph database layer
- **CouchDB** — Rebuilt as a layer over FDB

FDB Design Principles

1 Divide-and-Conquer

FDB decouples the Transaction Management System (write path) from distributed storage (read path), enabling independent scaling. Within the TMS, processes are assigned specialized roles: timestamp management, commit acceptance, conflict detection, and logging.

2 Make Failure a Common Case

All failures are handled through a single, unified recovery path. The transaction system proactively shuts down when it detects any failure, reducing all failure handling to one well-tested code path.

3 Fail Fast and Recover Fast

FDB minimizes Mean-Time-To-Recovery (MTTR). Total recovery time (detect + shutdown + recover) is typically less than 5 seconds in production, ensuring high availability despite failures.

4 Simulation Testing

FDB uses a deterministic simulation framework to test correctness under arbitrary fault combinations before deployment. This allows developers to simulate years of production failures in hours.

System Interface: A Minimal but Powerful API

Single Key Operations

```
get(key)
→ returns value of a single key

set(key, value)
→ writes the value of a key
```

Key Range Operations

```
getRange(begin, end)
→ returns sorted list of KV pairs in range

clear(begin, end)
→ deletes all KV pairs in range or by prefix
```

Transaction Semantics

- Writes (**set()**, **clear()**) are **buffered locally** until **commit()**
- Mutations update a snapshot of the DB at a certain version
- Changes are applied atomically when the transaction commits
- **Read-your-writes** semantics: uncommitted writes visible visible within the same transaction

Constraints

- Key size: ≤ 10 KB; Value size: ≤ 100 KB
- Transaction size: ≤ 10 MB (keys + values + conflict ranges)

FDB Architecture: An Unbundled Design

Control Plane

- **Coordinators:** Disk Paxos group for metadata persistence
- **ClusterController:** Monitors all servers, recruits singleton processes
- **Singleton processes:** Sequencer, DataDistributor, Ratekeeper

Data Plane

- **Transaction System (TS):** In-memory transaction processing (Sequencer + Proxies + Resolvers)
- **Log System (LS):** Write-Ahead-Log storage (LogServers)
- **Storage System (SS):** Data storage and read serving (StorageServers)

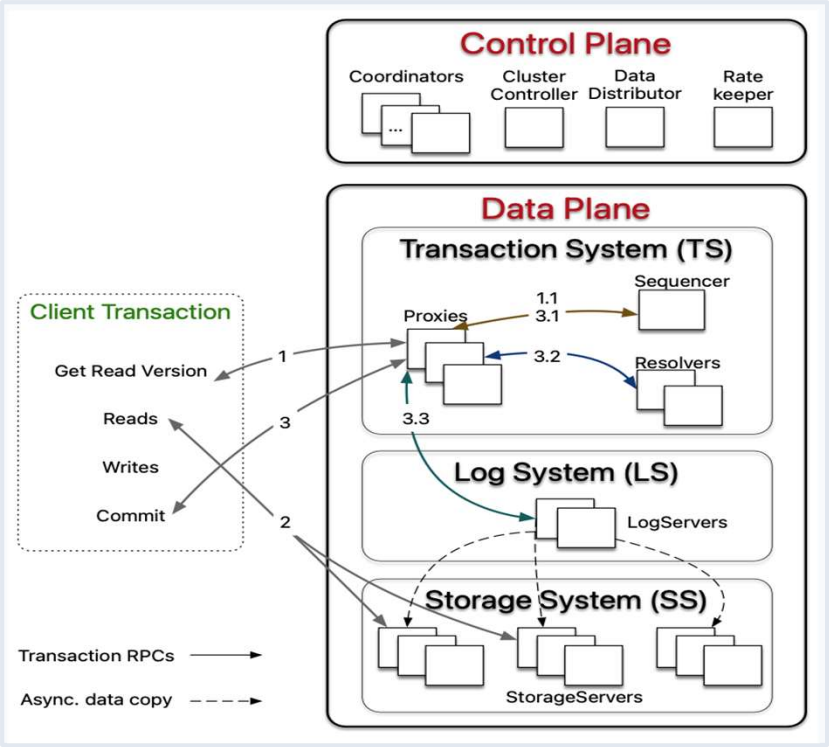


Figure 1: The architecture and transaction processing of FDB.
Source: Zhou et al., SIGMOD 2021.

Control Plane: Cluster Orchestration

Coordinators — The Foundation

Form a Disk Paxos group that persists critical system metadata.

Provide fault tolerance: majority must be alive for progress. Typically deployed with 5 or 9 replicas across failure domains.

ClusterController (CC) — The Orchestrator

Monitors all servers in the cluster continuously. Recruits and manages three singleton processes.

Singleton	Role
Sequencer	Assigns read and commit versions to transactions; recruits TS components
DataDistributor	Monitors failures; balances data shards among StorageServers
Ratekeeper	Provides overload protection; throttles clients when system is busy

> Fault Handling:All three singletons are automatically re-recruited if they fail or crash.

Data Plane: Three Decoupled Subsystems

Transaction System

WRITE PATH

All components are stateless (except Sequencer's version counter):

- **Sequencer**

  Assigns monotonically increasing read/commit versions; recruits Proxies, Resolvers, LogServers

- **Proxies**

  Provide MVCC read versions to clients; orchestrate transaction commits

- **Resolvers**

  Implement lock-free optimistic concurrency control; detect read-write conflicts

Log System

DURABILITY

- **LogServers**

  Replicated, sharded, distributed persistent queues

- **WAL Storage**

  Each queue stores Write-Ahead-Log data for a specific set of StorageServers

- **Replication**

  Synchronous replication: k = f+1 replicas for f-failure tolerance

Storage System

READ PATH

- **StorageServers**

  Store data shards; serve client reads directly

- **Async Pull**

  Asynchronously pull mutation logs from LogServers in the background

- **Storage Engine**

  Modified SQLite B-tree engine with MVCC support

Read-Write Separation Enables Independent Scaling

Scaling Reads

- Add more **StorageServers** to the Storage System
- Read throughput scales **linearly** with StorageServer count
- Clients read directly from StorageServers (no bottleneck through TS)

Scaling Writes

- Add more **Proxies, Resolvers, and LogServers**
- Write throughput scales with TS and LS capacity
- Singletons (ClusterController, Sequencer) are **NOT bottlenecks** — they only do metadata operations

Components & Scaling

Component	Scales	Bottleneck?
StorageServers	Reads	No — add more
Proxies	Write commits	No — add more
Resolvers	Conflict detection	No — add more
LogServers	WAL durability	No — add more
Sequencer	Version assignment	No — metadata only
ClusterController	Cluster monitoring	No — metadata only

Bootstrapping: FDB Needs No External Services

Self-Contained Design

FDB has no external service dependency — all metadata is stored within the system itself.

Metadata Storage Hierarchy

Data	Stored In
User data	StorageServers
StorageServer metadata	LogServers
LogServer configuration	Coordinators (all of them)

Bootstrapping Process

1. Coordinators (Disk Paxos group) elect a leader
2. Leader reads the TS configuration from Coordinators
3. New **Sequencer** is recruited by the ClusterController
4. Sequencer spawns new TS and LS instances
5. Proxies recover StorageServer metadata from old LogServers
6. New TS becomes ready to accept client transactions

Source: Zhou et al., SIGMOD 2021

Reconfiguration: Clean Slate Recovery via Epochs

Trigger Conditions

Reconfiguration is triggered when:

- Any Proxy, Resolver, or LogServer fails
- Database configuration changes (e.g., adding new servers)
- Sequencer process terminates for any reason

Reconfiguration Process

1. Sequencer process is terminated (proactively shuts down)
2. ClusterController detects Sequencer failure
3. CC recruits a new Sequencer
4. New Sequencer starts a **new epoch** via the recovery process
5. New epoch recruits fresh Proxies, Resolvers, and LogServers
6. New TS becomes ready to accept transactions

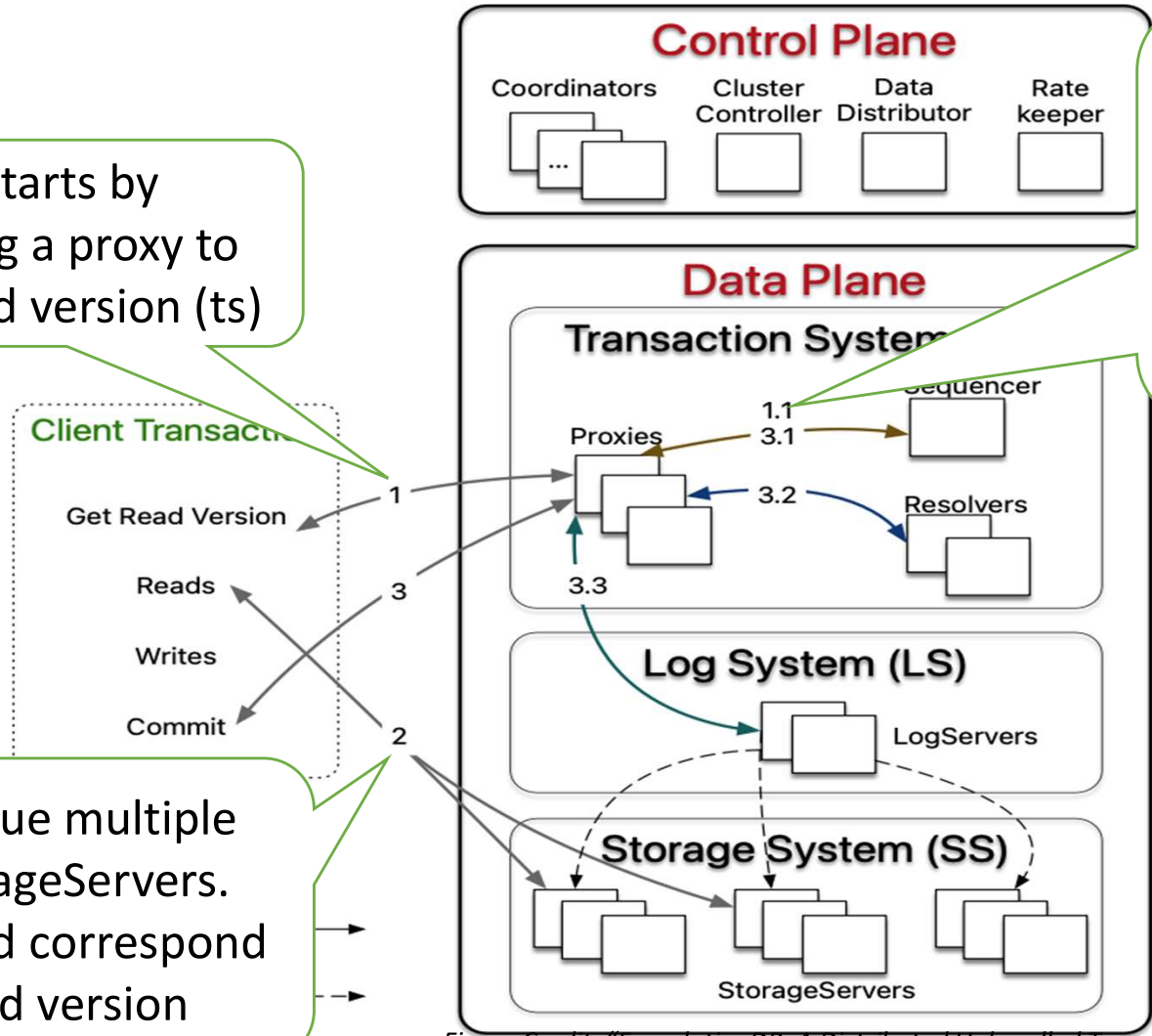
Key Insight

Each epoch has a unique Sequencer. Transaction processing is divided into epochs, where each epoch represents one generation of the Transaction Management System.

Transaction Management: End-to-end Trxn Processing (Read)

Trxn starts by contacting a proxy to obtain read version (ts)

Proxy asks Sequencer for read version (no less than prev. issued trxn commit version) & send it back to client



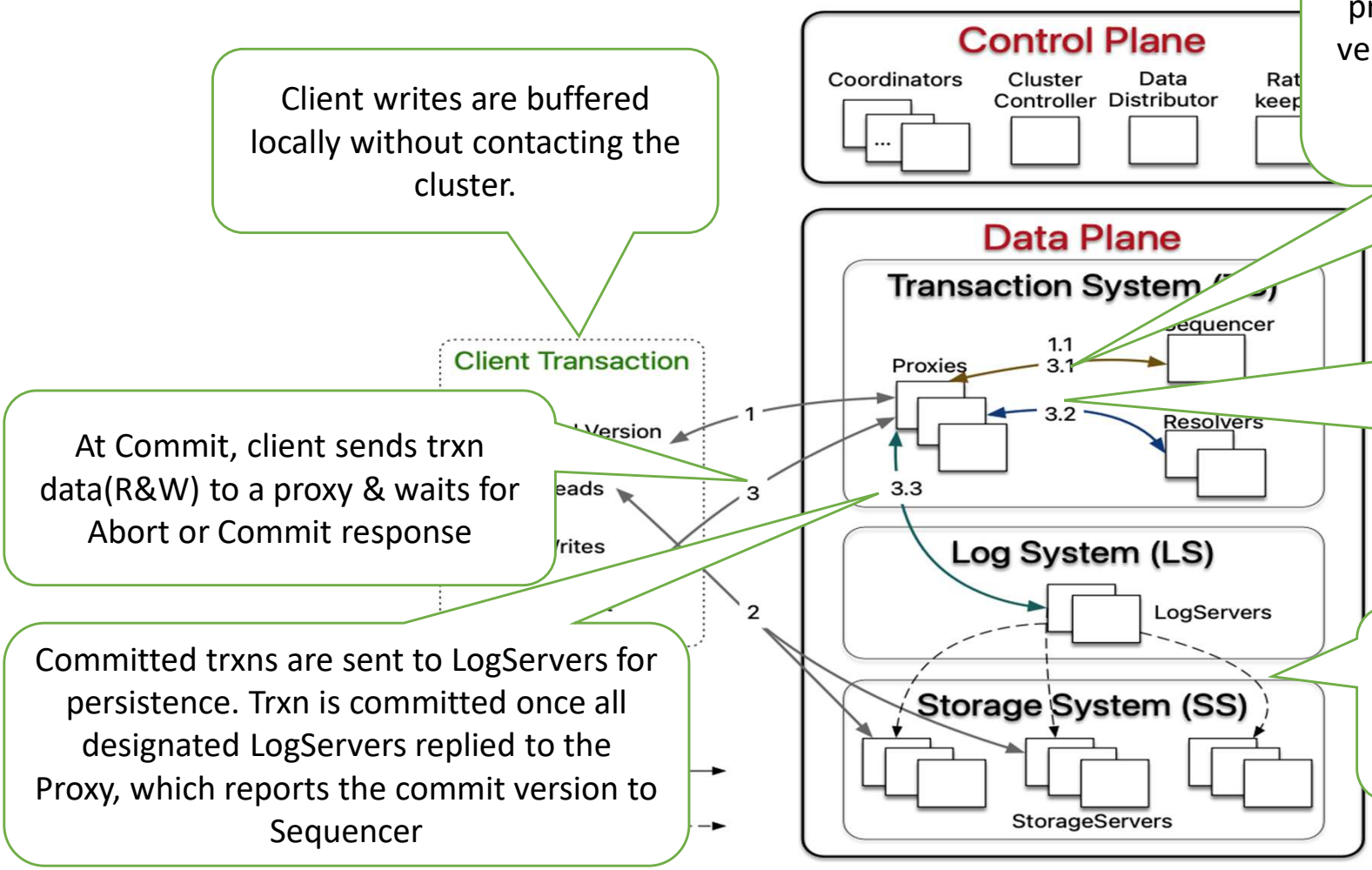
Client may issue multiple reads to StorageServers. Values returned correspond to their read version

Figure Credit: "FoundationDB: A Distributed Unbundled Transactional Key Value Store, by Zhou J, et al. 2021

Transaction Management: End-to-end Trxn Processing (Write/Commit)

Client writes are buffered locally without contacting the cluster.

Proxy asks Sequencer for commit version (no less than prev. issued trxn read/commit versions), advance its version & send trxn info to range-partitioned Resolver



At Commit, client sends trxn data(R&W) to a proxy & waits for Abort or Commit response

Resolver: implements optimistic con. ctrl. by checking for R/W conflicts. Trxn proceeds if no conflict. Marked as aborted, otherwise

Committed trxns are sent to LogServers for persistence. Trxn is committed once all designated LogServers replied to the Proxy, which reports the commit version to Sequencer

StorageServers pull mutation log from LogServers and apply committed updates to disk

Read-Only and Snapshot Reads: Relaxing Isolation for Performance

Read-Only Transactions

- Execute entirely at the **read version** (snapshot) obtained from the Proxy.
- Never conflict with other transactions since they do not not write data.
- **Commit locally** at the client without contacting the cluster.
- Zero overhead on Resolvers and LogServers, making them highly efficient.

Snapshot Reads

- Selectively relax isolation for specific reads within a read-write transaction.
- Do **not add** to the transaction's read conflict ranges.
- Prevent the transaction from being aborted due to concurrent writes to the snapshot-read keys.
- Ideal for background tasks, analytics, or when strict serializability is not required for certain keys.

"By selectively using read-only transactions and snapshot reads, applications can significantly reduce conflict rates and improve overall system throughput while maintaining strict serializability where needed."